Memory and Obsidian

Crew keeps durable memory in plain Markdown: your repository's contracts, ADRs and code map stay authoritative for that repository, and an Obsidian vault holds the lessons that cross projects. The `obsidian-vault` plugin owns that vault. It sets it up, captures sessions into it, gardens the captures into notes, and answers recall queries from it. Crew calls the plugin rather than doing any of this itself.

Every command below is `vault_ops.py`, which ships with the plugin. It is **dry-run by default**: it prints the exact change and writes nothing until you repeat it with `--apply`. Exit 0 means done or healthy, 1 means there is something to do or something wrong, and 2 means the command was refused or mistyped.

Set a shorthand once per shell. It picks the newest installed version of the plugin:

```
# Linux / macOS / Git Bash

VO="python3 $(ls -d ~/.claude/plugins/cache/*/obsidian-vault/*/ | sort -V | tail
-1)hooks/scripts/vault_ops.py"

# Windows PowerShell

$VO = (Get-ChildItem "$env:USERPROFILE\.claude\plugins\cache\*\obsidian-
vault\*\hooks\scripts\vault_ops.py" |

  Sort-Object { [version]$_.Directory.Parent.Parent.Name } | Select-Object -Last
1).FullName

function vo { python $VO @args }
```

On Windows, read `$VO ...` in the examples below as `vo ...`. Inside Claude Code you do not need either: `/obsidian-vault:init` runs the same steps and asks you before each one.

What Gets Captured

When a session ends, and before the context is compacted, a hook appends one line to the primary vault:

```
inbox/pending-reflect.<host>.md
```

```
- [ ] 2026-09-23 14:02 | SessionEnd | session=4f1c... | cwd=/repos/app |
transcript=/home/you/.claude/projects/.../4f1c....jsonl
```

That is all it records: when, which session, which folder, and where the transcript is. It never copies conversation text into the vault. Each machine writes its own `<host>` file, so two machines syncing one vault never write the same file. One line per session, whichever trigger fires first. The capture hook cannot block or slow a session; if it fails it says so on stderr and moves on.

Capture, import and the gardener write to the primary vault only. If the primary is not available (an unmounted drive, say), they write nothing and say why; capture reports it on stderr and the session carries on. They never fall back to a recall vault.

An older version wrote a single `inbox/pending-reflect.md`. That file is no longer written, but it is still read, so an existing backlog drains rather than being stranded.

How Gardening Works

The gardener turns queued sessions into notes. It reads each transcript, writes or extends concept and decision notes with sources, and appends a line to the day's daily note.

It runs **on one host, once a day, in bounded passes**:

- at most **5 items or 10 minutes** per run, whichever comes first;
- an item is acknowledged only after the note it produced exists, is non-empty, and was written during that item's run. If the model fails, times out, or names a file it did not write (including one that was already there), the item stays queued for the next run;
- acknowledgements go to `inbox/reflected.<host>.md`, so the queue files keep one writer each;
- a session whose transcript lives on another machine is left for that machine and does not use up one of the five slots;
- only the designated host runs it (`gardener.host` in the config), and one run at a time;
- with `--commit`, it commits only the files that run wrote plus its own acknowledgement file. It never sweeps in anything else you had staged. git, including the vault's own git hooks, runs inside the same 10-minute bound: a hook still running then is stopped and the commit is reported as not made. Leave `--commit` off if Obsidian Git already commits the vault.

Run a pass by hand with `/obsidian-vault:garden`, or:

```
$VO queue                # what is waiting

$VO garden-run           # one bounded pass (designated host only)
```

A large backlog drains in the same bounded batches:

```
$VO drain                        # dry run: backlog size, batch count, first batch

$VO drain --apply --batches 4    # up to 4 passes of 5, stops early if one
acknowledges nothing
```

Upkeep

When	Do
Daily, automatically	The scheduled gardener. Check `~/.claude/obsidian/gardener.log` now and then for lines that start `left queued`.
After a plugin update	Re-run `$VO schedule --os <yours>` and reinstall the unit it prints. The unit names the plugin's versioned folder, which the update replaces.
When a vault moves or you add one	`$VO adopt` to review roles.
When recall seems blind	`$VO recall --query "<words you expect>"` and check which vaults it names.
When another project's notes outrank this repo's	crew sends the main checkout's folder name as `--project`; if your notes' `project:` values differ, list them in the repo's `memory.recall.projects`.
When the bridge misbehaves	`/obsidian-vault:doctor`, then `/obsidian-vault:repair`. The bridge is optional; capture, recall, import and gardening all work on the files without it.
When the code changed	`/crew:onboard --refresh <subsystem>` for the repository's own

When	Do
a lot	code map. That map lives in the repository, not the vault.

Setup

Every case ends in the same place: exactly one **primary** vault, which receives captures and imports, and optionally some **recall** vaults that are read but never written.

Case 1: Obsidian Is Not Installed

Linux

```
$VO detect-obsidian
```

It checks snap (`/snap/bin/obsidian`), Flatpak (`md.obsidian.Obsidian`), the `obsidian` deb, AppImage locations and `PATH`, and answers `INSTALLED`, `MISSING` or `UNKNOWN`. `UNKNOWN` means none of those checks could run. It is not the same as missing, and install refuses it.

```
$VO install-obsidian              # prints e.g.: Planned install (snap): sudo snap
install obsidian --classic

$VO install-obsidian --apply      # runs exactly that command
```

Without snap it offers `flatpak install -y flathub md.obsidian.Obsidian`. For a deb or AppImage it tells you what to download from obsidian.md and does not fetch anything. Choose one with `--method snap|flatpak|deb|appimage`.

Windows

```
vo detect-obsidian

vo install-obsidian               # Planned install (winget): winget install --id
Obsidian.Obsidian -e ...

vo install-obsidian --apply
```

Then create the vault:

```
$VO create-vault --name memory --path ~/vaults/memory

$VO create-vault --name memory --path ~/vaults/memory --apply

$VO adopt --role memory=primary --apply
```

On Windows use a path such as `C:\vaults\memory`. Open the folder once in Obsidian (**Open folder as vault**) so Obsidian registers it too.

Success looks like: `detect-obsidian` prints `INSTALLED`, a second `create-vault ... --apply` prints `Nothing to do`, and `$VO adopt` lists `memory primary`.

Case 2: Obsidian with One Vault

```
$VO adopt

1 vault(s). Ask for a role for each: primary (exactly one; receives captures and
imports), recall (read for injection), ignore.



  notes                    unassigned /home/you/notes

$VO adopt --role notes=primary            # dry run: shows role and default changes

$VO adopt --role notes=primary --apply
```

Success looks like: running the same `--apply` again prints `Every requested role is already set. Nothing to change.` and exits 0.

Case 3: Several Vaults

`adopt` lists every vault in the config and every vault Obsidian itself knows about. Decide on each one separately:

- `primary`: this vault receives captures and imports. Choose exactly one.
- `recall`: this vault is searched for context but never written.
- `ignore`: leave this vault alone. The answer is stored so a re-run does not ask again.

Then write every answer in one call:

```
$VO adopt --role memory=primary --role work-notes=recall --role journal=ignore

$VO adopt --role memory=primary --role work-notes=recall --role journal=ignore --
apply
```

The command refuses any result with no primary, with two, or with any listed vault left without a role, and writes nothing when it does. That is why every answer goes in one call. To move the primary later, demote the old one in the same call: `--role memory=recall --role new-memory=primary`.

Success looks like: `$VO adopt` lists each vault with its role and no `[FAIL]` line.

Case 4: Importing a Folder

Bring notes from a plain Markdown folder, or from another vault by name, into the primary vault:

```
$VO import --source ~/old-notes                # dry run: counts and collisions

$VO import --source ~/old-notes --apply
```

On Windows, for example: `vo import --source C:\Users\you\Documents\old-notes --apply`.

Notes land under `imported/<folder name>/` with the same sub-folders. Each one gets two frontmatter keys: `imported_from` (the absolute source path) and `imported_at` (the UTC date). Line endings become LF. The import never overwrites a file. If a file already exists at the destination, the import skips it and reports it as a `COLLISION`. With `--suffix-collisions` it writes the new file beside it as `<name> (imported).md`; run again, it recognises that copy and writes nothing. A destination that passes through a symlinked folder, or lands outside the vault, is refused as `outside-vault`. Files that are not Markdown are counted and left behind. Use `--dest-subdir <folder>` to choose another destination inside the vault.

Success looks like: `Imported N note(s)`; running it again reports `already-imported: N` and writes nothing.

Scheduling the Gardener

On the one machine that should garden, run:

```
$VO schedule --os cron --designate           # Linux without a systemd user session
(root, containers)

$VO schedule --os systemd --designate        # Linux desktop with systemd --user

vo schedule --os windows --designate
```

Each command prints the unit, the install commands, a verify command and a remove command. It installs nothing. Paths are quoted for the shell that reads them, so a folder name with a quote, a space or `$` is safe to paste. Add `--apply` to make this machine the designated gardener, then run the printed install commands yourself. The default time is 02:23; change it with `--time HH:MM`.

Success looks like: after the first scheduled run, `~/.claude/obsidian/gardener.log` has `acked <session>` lines, and `$VO queue` shows fewer items.

The Ticket Board

The same vault can hold a Kanban board of your tickets, which crew moves for you at each lifecycle step. Crew writes the files itself, so nothing here needs the bridge or the `obsidian-vault` plugin; the vault needs `.obsidian/` and the Kanban community plugin.

Turning it on. In `.crew/config.json`, set `tracker` to `"obsidian"` and give the `obsidian` block a `vaultPath`, a `boardDir` (relative to the vault) and optionally `board` (default `Board.md`) and `columns` (your lane names). A repository that ran `/crew:migrate` also has `.crew/crew.json`, which holds the same settings as `tracker.kind` and `tracker.obsidian`; the tracker reads both and refuses when they disagree. Give every repository its own `boardDir`: with it unset, every repo shares one board at the vault root, and a card another repo owns is refused.

Ticket status	Lane (default name)
`direction`, `ready`, `needs-owner`	Backlog
`spec`, `planned`	Ready
`in-progress`	In Progress
`review`	Review
`done`	Done, checked, below

Ticket status	Lane (default name)
	`**Complete**`
`cancelled`, `superseded` (closed; leaving one needs `--reopen`)	Done, checked

Any other word in an INDEX status cell maps to no lane, and crew will not guess one. Moving to it is refused, and reading the ticket says the word is not a status crew knows rather than calling the card misplaced. For a word that was retired - `approved`, `merged`, `closed`, `new`, `parked` - it also names the word to write instead (`spec`, `done`, `done`, `direction`, `needs-owner`). Fix the INDEX cell by hand; crew never rewrites it for you.

Whose card it is. `/crew:brainstorm` writes a ticket note beside the board, `<boardDir>/T-0042.md`, once. Its `repo-id:` line is how crew tells your repository's cards from another's. The value is your origin URL with `.git` dropped and only the scheme and host case-folded, or the git directory's path when there is no origin. The user and the path keep their case, so `github.com:Team/App` and `github.com:team/app` are two repositories. An ssh origin keeps its username (`git@github.com:Team/app`); other URLs lose any user or token; a `file://` origin is decoded to its path, with `localhost` (or, on POSIX, any host) dropped. A quote around the value counts only as a matched pair.

Claiming an older note. A card whose note has no `repo-id:` (a board from crew 0.20, or a card you added by hand) is refused until you add one. The refusal names the exact line: add `- repo-id: <the id it prints>` to the note and run the command again. Crew never rewrites a note for you. A note written by an older crew for an origin with capitals in it holds the id lowercased; crew refuses it as another repo's and names the line to change it to.

The full rules, including every refusal, are in the crew README's former section 13c ("Optional: an Obsidian Kanban board"), now `docs/crew/reference/13c-optional-an-obsidian-kanban-board.md`.

Native Memories as Vault Pointers

Claude Code keeps its own native memory, one Markdown file per fact under `~/.claude/projects/<project>/memory/`. Crew can read such a file whose body is a single pointer line into your vault, with the frontmatter left as it is:

```
vault: <name> | note: <vault-relative path, forward slashes, ending .md>
```

The vault is named, not given as a path, so the same file works on every machine the vault syncs to. The name is looked up in that machine's `~/.claude/obsidian/config.json` (`vaults.<name>.path`; a vault with role `ignore` is not looked up). Only the name `memory` falls back: to crew's `memory.vaultPath`, then, only when the Obsidian config has no `vaults` block, to its legacy top-level `vaultPath`. `OBSIDIAN_VAULT_PATH` is not used. The line is exact and must be the whole body: a first line that starts `vault` and `:`, in any case, indent or spacing, is a pointer attempt when `note:` or `|` is on that line, or the next line starts with `|` or `note:` (a pointer wrapped before its `|`), or the line is a bare vault name alone (`vault: work`) or nothing after the colon (`vault:`); a table or a `Note:` line further down does not count; anything less than the full grammar is then `malformed`. With neither the line is prose: `Vault: keep client notes in the work vault, not personal.` is a memory, not a pointer. A config file counts as missing only when it is not there at all; one that is there but does not read, parse or have the expected shape (a `vaults` object of objects with a string `path`, a string `vaultPath`, crew's `memory` an object with a string or null `vaultPath`; no duplicate key, not nested too deep, at most 1 MiB) is `no-vault-config`, naming the field. A bad Obsidian config stops every name; a bad crew config stops `memory`, the one name it can answer for, and any name when there is no Obsidian config to say which failure applies.

```
python3 plugin/crew/hooks/scripts/crew_memory.py resolve --file <memory file>

python3 plugin/crew/hooks/scripts/crew_memory.py check --memory-dir <memory dir>
```

`resolve` prints `state:` and, when the note is found, `path:`. `check` prints one row per memory file (not `MEMORY.md`) and a count per state. Both read only; `--json` gives the same rows as JSON.

state	meaning	exit
`resolved`	the note exists on this host	0
`full-text`	not a pointer; the body is the memory	0
`malformed`	a `vault:` first line, in any case, indent or spacing, that is not a valid pointer alone in the body (absolute path, `:`, `..`, a second field, an invisible character)	1

state	meaning	exit
`no-vault-config`	no Obsidian config and no `memory.vaultPath`, or a config file that cannot be read, does not parse or has a field of the wrong shape	1
`vault-unknown`	this machine names no vault of that name, or marks it `ignore`	1
`vault-unavailable`	the configured folder is not there (not mounted, not synced)	1
`note-missing`	the vault is there, the note is not	1
`outside-vault`	the path passes through a symlink inside the vault, or leaves it	1
`unreadable`	the memory file is not a regular file readable as UTF-8 (`check` never opens a FIFO, device or directory), or a folder or note inside the vault cannot be read or opened	1

An unavailable vault is never replaced by another one that happens to hold a note at the same path.

Saving a Memory as a Pointer

`save` turns one memory that holds its full text into a pointer, note first:

```
python3 plugin/crew/hooks/scripts/crew_memory.py save --file <memory file> --tag <tag>

python3 plugin/crew/hooks/scripts/crew_memory.py save --file <memory file> --tag <tag> --apply
```

Without `--apply` it prints the plan (the vault, the note, `create`, `append` or `unchanged`, and the pointer line) and writes nothing. Read your vault's own `CLAUDE.md` for its folders and tags first; `--note`, `--title`, `--type` (`concept` by default) and `--project` change the defaults (`memories/<project>/<title>.md`, the title taken from the memory's `name:` line).

It writes only to the one writable vault: the vault with role `primary` (without roles, the `default: true` vault, else the first; with no `vaults` block, `memory`). The folder must hold `.obsidian/`. A `recall` or `ignore` vault is never written, and a primary that is not mounted is reported, not replaced. The note gets the vault's six frontmatter keys plus `project` and `memory_id`; saving the

same memory again appends a dated `## Update` passage, and a note that belongs to another memory is a `collision`.

The order is the point. The note is written through a temp file, read back, and the pointer resolved; only then is the memory's body replaced, its frontmatter kept byte for byte, through a temp file and a rename. Any refusal or failure prints `kept-full-text: <reason>` and leaves the memory exactly as it was:

state	memory file	exit
`pointer-written`	body replaced by the pointer	0
`already-pointer`	untouched	0
`kept-full-text: no vault configured`	untouched	0
`kept-full-text: vault unavailable`, `no primary`, `several primaries`, `not a vault`, `config unreadable`	untouched	1
`kept-full-text: collision`, `ascii-required`, `outside-vault`, `bad-note-path`, `the existing note is not UTF-8`	untouched	1
`kept-full-text: MEMORY.md is the index`, `the memory file is a symlink`, `the memory has no body to save`	untouched	1
`kept-full-text: another save is running now`, `lock failed`, `note write failed`, `the note changed during save`, `note not readable after write`, `the memory file changed during save`, `the memory file cannot be read again`, `pointer write failed`	untouched	1

Two `save` runs by the same user on the same machine do not interleave: each holds kernel locks for the note and for the memory, on files in `~/.cache/crew/memory-locks` (`$XDG_CACHE_HOME` when set; `%LOCALAPPDATA%\crew\memory-locks` on Windows), never in the vault. The operating system drops a lock when its save exits or is killed, so `another save is running now` means one is running; a lock that cannot be taken at all (an unwritable cache, a full disk, or no absolute cache folder because `HOME` is unset) is `lock failed`. A file is locked by its real path, case-folded, and by its inode, so a symlinked folder, `..`, a case variant and a hard link all meet the same lock. The lock does not cover a save on another machine that syncs the same vault, or an edit by

Claude Code, Obsidian or any other program, none of which take it. For those, the memory and an existing note are compared again right before each rename. An edit by another program in the instant between that compare and the rename is not detected: a rename cannot compare and swap. An existing note keeps every byte, a BOM and CRLF line endings included; only its `updated:` value changes and the passage is appended. A new note gets mode 0644 less your umask.

Claude Code 2.1.289 was seen to keep a one-line pointer body across new sessions (it rewrites the frontmatter when it updates the memory, and kept the pointer). If a later version rewrites the body with full text, `check` shows it as `full-text` again; run `save` again.

Converting Existing Memories

`migrate` runs `save` over every memory in one folder. It never runs by itself, and without `--apply` it writes nothing:

```
python3 plugin/crew/hooks/scripts/crew_memory.py migrate --memory-dir <memory dir>
--tag <tag>

python3 plugin/crew/hooks/scripts/crew_memory.py migrate --memory-dir <memory dir>
--tag <tag> --apply
```

The preview has one row per memory file, in name order; `MEMORY.md` is never a row and never edited:

action	meaning
`convert`	`save` would create the note shown
`append`	a note with this `memory_id` already exists; `save` would add to it
`skip: already a pointer`	nothing to do
`skip: <state>`	a pointer that does not resolve, or a `malformed` one
`refuse: <reason>`	anything `save` refuses, a title that is not a portable file name (`: ? * < > | " \`, a trailing dot or space, a device name such as `CON`), two files that would write one note (`duplicate note path`), a note that belongs to another project (`note belongs`

action	meaning
	`to another project`), or a folder named like a memory (`not a file`)

With `--apply`, each `convert` and `append` row is saved one file at a time. A file whose save fails is shown as `failed` with `save`'s `kept-full-text` reason, and the others still go ahead. One `--tag` set covers the run; use `--only <file name>` to run batches with different tags. `--note-dir`, `--type` and `--project` work as they do for `save`; without `--project`, a folder `~/.claude/projects/<slug>/memory` uses `<slug>` as the project, so two projects' memories never land in one note. Nothing is recorded between runs, so running it again converts nothing twice.

`restore` is the way back for one memory:

```
python3 plugin/crew/hooks/scripts/crew_memory.py restore --file <memory file> --
apply
```

It puts the note's text back as the memory's body (frontmatter kept) and leaves the note alone. Without `--apply` it shows the first line and the line count. A pointer that does not resolve is reported and nothing is written.

Confirming Recall Reaches Your Sessions

(written by the context-hook ticket)

Path-Scoped Rules from the Code Map

The code map also reaches a session with no hook at all, as `.claude/rules/<subsystem>.md` files:

- **Generated by `/crew:onboard` and `/crew:migrate`.** After it writes or refreshes a note, `/crew:onboard` (including `--refresh <subsystem>`) runs `crew_instructions.py rules`. `/crew:migrate` lists the rules in its preview and writes them after `--apply`. Each rule is at most 30 lines: a `paths:` list, the note's landmines, and the sha256 of the note it came from. A hand-written file already at that path is reported and left alone.
- **Loaded by Claude Code when a matching path is touched.** Claude Code's memory docs say a `paths:` rule loads when Claude reads a file matching one of its patterns, so a subsystem's landmines arrive only in sessions that touch that subsystem. That is from the docs; this guide has not measured it.

- **Checked for drift in CI.** `scripts/check_instructions.py` runs `crew_instructions.py rules --check`. A note edited without regenerating its rule fails as `stale`. A hand-written file at a generated path fails under its own label, not as drift. Commit the rules with the notes, and re-run `/crew:onboard --refresh <subsystem>` to fix drift.

Confirming Recall Reaches Your Sessions

crew's context hook (`plugin/crew/hooks/scripts/crew_context.py`) puts two kinds of memory into a session: **code-map slices** from `.crew/codemap/`, and **vault recall** from the Obsidian vaults `obsidian-vault` manages. This page shows how to prove, on your own machine, that recall reaches the main session *and* the subagents it dispatches, and that the model actually uses it.

A third channel needs no hook: `/crew:onboard` and `/crew:migrate` generate `.claude/rules/<subsystem>.md` from the same code map, and Claude Code loads each rule when a file matching its `paths:` is read. This page does not test that channel; the drift check in `scripts/check_instructions.py` covers whether the rules match their notes.

Which Channel Reaches Which Agent

Measured on Claude Code 2.1.281 on 2026-09-23 with a canary per hook event (one `claude -p` run, Haiku, a `general-purpose` subagent told to report what was in its context without using tools):

Hook event	Reaches the main agent	Reaches a subagent
`SessionStart`	yes	**no**
`UserPromptSubmit`	yes	**no**
`SubagentStart`	n/a	**yes**

The subagent's verbatim answer in that run: `Harbour colour: UNKNOWN`, `Orchard number: UNKNOWN`, `Lighthouse keeper name: OSWIN-TARRANT-92`, where only the last came from `SubagentStart`. So crew registers the hook on `SubagentStart` as well. That event's payload names the agent (`agent_id`, `agent_type`) but does not include the task, so the hook reads the task back from the parent transcript's latest `Agent` tool call. Commands that write a subagent prompt themselves can also paste in a slice:

```
python3 "<crew plugin root>/hooks/scripts/crew_context.py" --slice-for-subagent \
  --query "<the task text>" --paths <files the task touches>
```

Where Recall Comes From

- The vaults and their roles come from `~/.claude/obsidian/config.json`. Each vault has a `role`: `primary`, `recall` or `ignore`. A vault with no role is `primary` if it is the default and `recall` otherwise. crew **never** asks an `ignore` vault, even if the repo lists it.
- A repo can set its own order in its crew config: `memory.recall.vaults` (an ordered list of names) and `memory.recall.maxChars` (default 800). Snippets come back in that vault order first, then in the order the search ranked them.
- Each recall names the repo's project, so that notes of this project rank first inside each vault: `memory.recall.projects` (a list of names, repo-only) when set, otherwise the main checkout's directory name (a linked worktree reports the main checkout's name). crew sends it as `--project=<a,b>` and ranks nothing itself. An obsidian-vault CLI too old to know `--project` exits 2; crew then asks once more without it, inside the same 4-second budget, and the log's `recall` record says so: `project` is the list sent, `projectUsed` is `true` when the answer came from the call with the project, `false` when it came from the retry without it, and `null` when no project was sent or no call answered with JSON. Set `memory.recall.projects` when the vault's `project:` values do not match the checkout's folder name.
- crew calls obsidian-vault's read-only CLI: `vault_ops.py recall --query ... --vaults a,b --max-chars N --json [--project=a,b]`. If the plugin is missing, or the CLI fails, times out or prints something other than JSON, the session goes on without recall and the log records a miss with the reason.
- An empty recall can be the CLI working as meant: it skips stop words, needs more than one matching word for a query of three or more words, matches whole words only, and never reads `wiki/sessions/archive/`. To check whether a note is there but held back, run the CLI with `--min-terms 1` (no floor) and `--include-excluded` (the archive too).
- Every recall line starts with the vault's name: `- [vault:<name>] <note path>: <text>`. Any snippet the CLI returns without a vault name is dropped.

The Seeded-Note Proof

1. Make a scratch vault holding one note whose wording appears nowhere else, for example `notes/tidewater-ledger.md`: "The Tidewater ledger reconciliation cutoff is 03:17 Kestrel time, and the approving steward is Marisol Quennell-Vantage."
2. Add that vault to your obsidian config with `"role": "recall"`. For a dry run that leaves the real config alone, write a copy and point `CREW_OBSIDIAN_CONFIG` at it.
3. In a scratch project that has a `.crew/` directory, register crew's hook in `.claude/settings.json` for `SessionStart`, `UserPromptSubmit` and `SubagentStart`. Use the command `bash "<crew plugin root>/hooks/scripts/crew-context.sh"`. `crew_instructions.py claude-hooks` prints the full set of entries.
4. Start a fresh session and ask for a subagent that may not use tools:

```
claude -p --model haiku --output-format stream-json --verbose --include-hook-events
\

  "Use the Task tool once with exactly this prompt: 'Without using any tools, answer
only from \

your context: what is the Tidewater ledger reconciliation cutoff, and who is the
approving \

steward? If it is not in your context, answer UNKNOWN. Name the source label.' Then
reply with \

the subagent's answer verbatim."
```

What success looks like. The `Agent` tool call's prompt does not contain the answer, the `SubagentStart` hook response contains `- [vault:ops-recall] notes/tidewater-ledger.md: ... 03:17 Kestrel time ... Marisol Quennell-Vantage`, and the subagent answers with the answer and cites the label, with `tool_uses: 0`. From the 2026-09-23 run, verbatim:

```
Based on the context provided:



**Tidewater ledger reconciliation cutoff:** 03:17 Kestrel time



**Approving steward:** Marisol Quennell-Vantage
```

```
**Source label:** [vault:ops-recall]
```

That run used a stand-in for obsidian-vault's `recall` command: a word-overlap search over the scratch vault's files, implementing the same contract. It has since been re-run against the real command; see below.

Measured against the real `vault_ops.py recall`, 2026-09-23

Claude Code 2.1.281, `claude -p --model haiku --no-session-persistence --setting-sources project --strict-mcp-config`, crew 0.20.19 with `memory.inject: true`, in a scratch project and scratch vault. obsidian-vault's own CLI (`plugin/obsidian-vault/hooks/scripts/vault_ops.py`) was named with `CREW_VAULT_OPS`, and both tools read a scratch obsidian config through `CREW_OBSIDIAN_CONFIG` and `OBSIDIAN_VAULT_CONFIG`. The seeded note: "The Brackenmoor sluice-gate audit window opens at 04:41 Corvid time, and the countersigning warden is Ysolde Pemberton-Karrow." The `Agent` call's prompt asked the question and did not contain the answer. The subagent answered with `tool_uses: 0`, verbatim:

```
Based on the vault recall in my context:



**Time:** 04:41 Corvid time

**Countersigning Warden:** Ysolde Pemberton-Karrow

**Source:** [vault:proof-recall] notes/brackenmoor-sluice.md
```

The `SubagentStart` line in the emission log:


```
{"agent_id": "a74ce527b5ab3e7cb", "agent_type": "general-purpose", "budget": 2000,
"chars": 423, "dedupHits": 0, "event": "SubagentStart", "harness": "claude",
"lines": 4, "query_from": "last-prompt", "recall": {"dropped": 0, "reason": "",
"snippets": 2, "status": "hit", "vaults": ["proof-recall"]}, "session": "08038c78-
3372-4ccb-85d0-2d765b8c79db", "sources": [{"kind": "vault", "note":
"notes/other.md", "vault": "proof-recall"}, {"kind": "vault", "note":
"notes/brackenmoor-sluice.md", "vault": "proof-recall"}], "truncated": false, "ts":
"2026-09-23T20:59:07Z"}
```


`query_from: last-prompt` means the recall query came from the session's last user prompt, not from the `Agent` call in the transcript. `--no-session-persistence` leaves no transcript to read

back. So this run proves that the real CLI's recall reaches a subagent. It does not prove that the query is taken from the `Agent` call.

Reading `--stats`

Every emission adds one JSON line to `<git-common-dir>/crew/context-log.jsonl`, which all worktrees of the repo share. Outside git, the log is `.work/crew/context-log.jsonl`. To summarise it:

```
python3 "<crew plugin root>/hooks/scripts/crew_context.py" --stats          # add --
json for machines
```

This is the output after the proof run above, without its first line (the log path):

```
emissions: 3  injected chars: 647  dedup hits: 0  truncated: 0

  SessionStart: 1 emissions, 45 chars

  SubagentStart: 1 emissions, 352 chars

  UserPromptSubmit: 1 emissions, 250 chars

vault recall: 2 hit, 1 miss, 0 skipped; 2 snippets injected; code-map slices: 0

  miss reasons: no-hits=1

vault MCP/search tool calls seen: 0

by harness: claude=3
```

- **hit / miss / skipped** are counted per recall attempt. `cli-missing` means obsidian-vault is not installed where crew can find it. `cli-exit-N`, `cli-timeout` and `cli-bad-json` mean the CLI ran and failed. `no-vaults` means no vault is `primary` or `recall`. `no-hits` means the search matched nothing.
- **dedup hits** count slices that were not injected again because this context already had them. Deduplication applies per subsystem or vault note, and per compaction epoch: after `/compact` or `/clear`, crew injects them once more.
- **vault MCP/search tool calls seen** counts the times the model searched a vault itself (`mcp__obsidian-*`, `mcp__basic-memory__*`). The baseline before crew 1.0 was 5 calls in 305 sessions. If recall is helping, this number stays low while hits rise.

- **by harness** separates Claude Code from Codex (`codex`) and from dispatch slices (`cli`).